

College of Engineering and Technology
Web Engineering Department

PROJECT DOCUMENTATION

FreshMart Web

Project Title: FreshMart Web - Online Supermarket Website

Supervised By: [Dr. / Amr Fahmy]

Course: Web Engineering

Semester: [Semester 8]

Academic Year: 2025 / 2026

Website Url: <https://freshmartproject.onrender.com/>

Github Url: <https://github.com/Amr-Khaled2004/FreshMartProject.git>

Table of Contents

1. Project Team
2. Introduction & Problem Statement
3. Project Objectives
4. Technology Stack
5. System Architecture & Design
6. Project Steps & Phases
7. Group Task Distribution
8. Development Timeline
9. Full Project Documentation
10. Submission Checklist

1. Project Team

1.1 Team Members

Full Name	College ID	Role / Responsibility	Email
Nour Shafie	[221005721]	Frontend: home page, product catalog/cards, search/category/sort, Add to Cart stock validation; deployment and documentation	nourshafie22@gmail.com
Dania Yasser	[221004605]	Frontend: checkout form, payment validation, order placement UI, order history, responsive review	daniagamal0@gmail.com
Talia Hussein	[221006502]	Frontend: login/register, user session, token storage, dynamic navbar, cart page controls	taliaalnayef3@gmail.com
Amr Khaled	[221004748]	Backend: Express setup, MongoDB connection, User model, register/login APIs, JWT, route protection	amrksayed200@gmail.com
Ziad Tarek	[221006730]	Backend: product/order APIs, stock changes, admin management, cancellation stock restore, email/outbox logic	ziadghamry6@gmail.com

1.2 Sub-Group Distribution

Sub-Group	Member(s)	Module / Layer	Key Deliverable
Frontend	Nour Shafie, Talia Hussein, Dania Yasser	UI pages, user flow, cart, checkout, and responsiveness	Home page, product catalog/cards, search/category/sort, login/register, session/token handling, dynamic navbar, cart controls, checkout, order history, and responsive review
Backend / Database	Amr Khaled	Express setup, MongoDB connection, user model, authentication, route protection	Configured Express backend, connected MongoDB, created User model, implemented register/login APIs, JWT authentication, and protected routes
Backend / Admin & Orders	Ziad Tarek	Product/order APIs, stock logic, admin management, cancellation, outbox logic	Implemented product/order endpoints, stock updates, admin product/order management, cancellation stock restore, and email/outbox logic
DevOps / QA / Documentation	Nour Shafie + all members	Render deployment, testing, documentation, presentation, demo support	Hosted the project using Render, tested main flows, finalized the report/slides, and prepared the live demo

2. Introduction & Problem Statement

2.1 Background

FreshMart Web is an online supermarket web application designed to simulate a real e-commerce grocery experience. The application allows customers to browse grocery products, create accounts, log in securely, add items to cart, place orders, and track their orders. It also includes an admin side for product and order management. The project demonstrates core Web Engineering concepts including client-server architecture, REST APIs, authentication, authorization, data persistence, and deployment.

2.2 Problem Statement

- Customers need a simple online supermarket interface to view available products and order groceries remotely.
- Manual product and order handling can be slow and error-prone without a centralized management interface.
- A supermarket website requires secure login, role-based access, stock handling, and organized order processing.
- The project needs an integrated frontend and backend that can be deployed online for a live demo.

2.3 Motivation

The motivation behind FreshMart Web is to build a realistic web application that connects user experience with backend business logic. It helps the team practice how modern websites handle authentication, product data, cart/order flows, admin control, and production deployment.

3. Project Objectives

- Build a multi-page online supermarket website with customer and admin interfaces.
- Implement secure user registration and login using hashed passwords and JWT authentication.
- Create RESTful API endpoints for products, orders, authentication, and admin actions.
- Store and manage project data using MongoDB collections for users, products, and orders.
- Integrate frontend pages with backend APIs for products, cart, checkout, and order management.
- Deploy the project online using Render and prepare a final report and presentation for a five-member team.

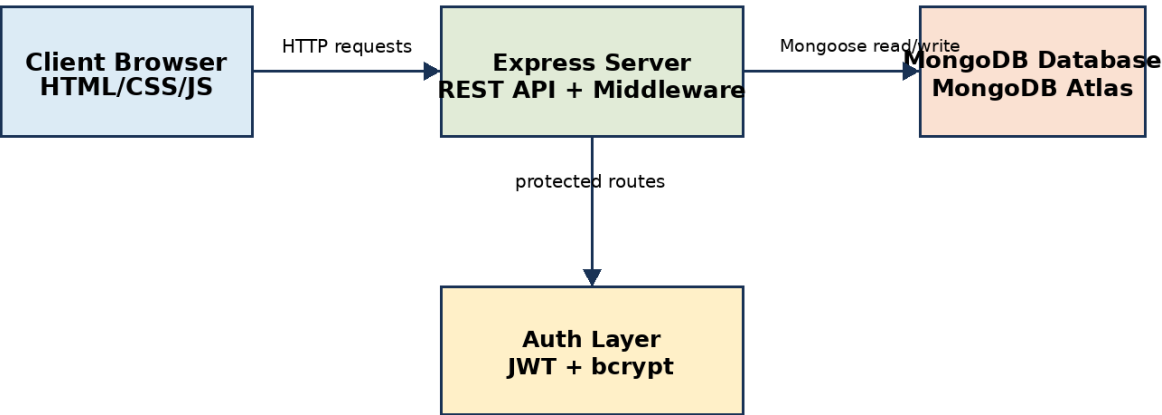
4. Technology Stack

Layer	Technology / Tool	Purpose / Notes
Frontend Framework	HTML5, CSS3, JavaScript	Client-side pages, DOM manipulation, product display, cart interactions
Styling	CSS / responsive layout techniques	Visual design, grids, forms, navigation, and responsive behavior
Backend Framework	Node.js + Express.js	Server-side logic and REST API endpoints
Database	MongoDB / MongoDB Atlas	Persistent database storage for users, products, and orders
ORM / ODM	Mongoose	ODM for defining schemas/models and interacting with MongoDB collections
Authentication	JWT + bcryptjs	Token-based sessions and secure password hashing
Middleware	cors, express.json, custom protect/adminOnly	CORS handling, JSON parsing, protected routes, role-based access
Version Control	Git + GitHub	Source code management and deployment connection
Deployment	Render	Single live deployment for the Express backend and frontend using Render
Testing	Postman, browser testing, Chrome DevTools	API checks, manual UI testing, responsiveness checks

5. System Architecture & Design

5.1 High-Level Architecture

The application follows a client-server architecture. The browser loads the frontend pages and sends HTTP requests to the Express server. The server handles authentication, authorization, business logic, and data operations. The database layer is MongoDB, which stores users, products, and orders in structured collections for persistent project data.



5.2 Frontend Architecture

- Pages: index/home, login/register, cart, checkout, admin dashboard, and supporting product pages.
- State approach: browser local storage/session data plus API responses for logged-in user actions.
- Routing approach: multi-page static website using HTML file navigation and JavaScript event handling.

5.3 Backend Architecture

- REST API implemented using Express.js and connected to MongoDB using Mongoose.
- Middleware includes CORS, JSON parsing, JWT protection, and admin-only access control.
- Main route groups: authentication, products, customer orders, and admin orders.
- Business logic includes product retrieval from MongoDB, product creation/deletion, placing orders, checking stock, and updating order status.

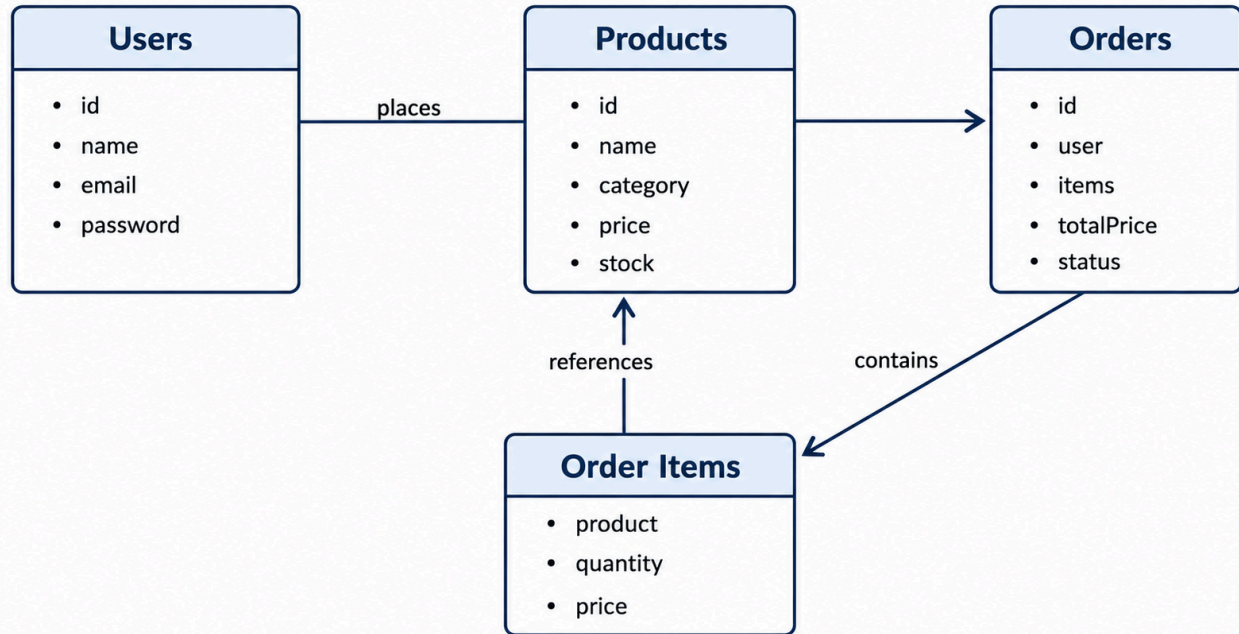
5.4 Database Schema

MongoDB is used as the main database instead of the previous database.json file. The backend connects to MongoDB using Mongoose and stores application data in separate collections.

Main collections:

Collection / Model	Main Fields	Purpose
Users	name, email, password, role, createdAt	Stores customer/admin accounts and authentication data
Products	name, category, price, image, stock, description	Stores supermarket product catalog and inventory information
Orders	userId, items, totalPrice, address, paymentMethod, status, createdAt	Stores customer orders and supports admin order tracking

Relationships: each order references the user who placed it, and each order item references a product. This keeps the system organized and allows the admin dashboard to read product and order data directly from MongoDB.



6. Project Steps & Phases

Phase	Step / Task	Expected Output / Deliverable
Phase 1	Project planning, scope definition, page list, and feature selection	Approved project idea and required features
Phase 2	Database structure design for users, products, and orders	MongoDB collections, Mongoose schemas, and sample seed data
Phase 3	Environment setup, GitHub repository, folder structure, and MongoDB connection setup	Initialized frontend/backend folders and configured MongoDB connection
Phase 4	Backend development: authentication, product API, order API, admin logic	Working Express endpoints
Phase 5	Frontend development: pages, forms, product cards, cart, checkout, admin	Functional user interface connected to the API
Phase 6	Security and role-based access	JWT login, bcrypt password hashing, admin-only routes
Phase 7	Testing and bug fixing	Verified main user/admin flows
Phase 8	Render deployment and final delivery	Live Render website, documentation report, and presentation slides

7. Group Task Distribution

Module / Feature	Assigned Member(s)	Sub-Group	Status
Home Page & Product Catalog	Nour Shafie	Frontend	Completed
Product Cards, Search, Category & Sort	Nour Shafie	Frontend	Completed
Login/Register, Session & Dynamic Navbar	Talia Hussein	Frontend	Completed
Cart Page Controls & Add-to-Cart Stock Validation	Nour Shafie + Talia Hussein	Frontend	Completed
Checkout, Payment Validation, Order Placement UI & Order History	Dania Yasser	Frontend	Completed
Express Setup, MongoDB Connection, User Model & Auth APIs	Amr Khaled	Backend / MongoDB	Completed
JWT, Route Protection & Protected Middleware	Amr Khaled	Backend / Security	Completed
Product/Order APIs, Stock Changes, Admin Management, Cancellation Restore & Email/Outbox Logic	Ziad Tarek	Backend / Admin	Completed
Render Deployment, Documentation, QA & Demo Support	Nour Shafie + all members	DevOps / QA	Completed

8. Development Timeline

Week	Phase / Task	Responsible Sub-Group	Deliverable
1	Kickoff, requirements, pages, and feature planning	All	Project scope and feature list
2	Database design and API contract	Backend + Database	Database schema and endpoint list
3	Environment setup, GitHub workflow, folder structure	All	Working local development environment
4	Backend API development: auth, products, orders	Backend	Core backend endpoints
5	Frontend development: layout, product cards, cart, checkout	Frontend	Core pages and UI components
6	Frontend-backend integration and role-based access	All	Connected application
7	Testing, bug fixing, and responsiveness checks	QA + All	Tested website and fixed bugs
8	Render deployment, final report, and presentation preparation	DevOps + All	Live Render demo app, report, and slides

9. Full Project Documentation

9.1 Features & Functional Requirements

- User registration and login with hashed password storage.
- JWT-based authentication for protected customer and admin endpoints.
- Product catalog display for customers.
- Cart and checkout flow for placing orders.
- Order creation with item quantities, total price, address, payment method, and status.
- Admin-only product management: add and delete products.
- Admin-only order management: view all orders and update order status.
- Role-based authorization to separate customer and admin features.

9.2 Non-Functional Requirements

- Performance: lightweight static frontend and MongoDB-backed API suitable for project demo use and persistent data storage.
- Security: passwords hashed with bcryptjs; protected APIs require JWT; admin routes require admin role.
- Usability: clear page navigation, product cards, forms, and admin actions.
- Responsiveness: pages should be tested on desktop and mobile widths using browser developer tools.
- Maintainability: separate frontend and backend folders; readable API structure; reusable middleware; organized Mongoose models for users, products, and orders.

9.3 API Documentation

Endpoint	Method	Description	Auth Required	Response
/api/auth/register	POST	Register a new customer account	No	201 Created + user data
/api/auth/login	POST	Login and receive JWT token	No	200 OK + token + user
/api/products	GET	Get all products	No	200 OK + products list
/api/products	POST	Add a new product	Admin	201 Created + product
/api/products/:id	DELETE	Delete a product	Admin	200 OK message
/api/orders	POST	Place a new customer order	Yes	201 Created + order
/api/orders/my-orders	GET	Get orders for the logged-in customer	Yes	200 OK + orders
/api/orders	GET	Get all orders for admin dashboard	Admin	200 OK + orders
/api/orders/:id/status	PUT	Update order status	Admin	200 OK + updated order

9.4 UI/UX Design Notes

- Design language: fresh supermarket/e-commerce theme with product cards, navigation, cart, and checkout pages.
- Color direction: green accents to reflect grocery/fresh products, supported by clean white backgrounds and readable dark text.
- Typography: standard web-safe fonts with clear headings, labels, and buttons.
- Key user flow: Home/Product browsing -> Login/Register -> Add to Cart -> Checkout -> Order history.
- Admin flow: Admin login -> Dashboard -> Manage products -> View/update orders.

9.5 Testing Plan

Test Type	Tool / Method	Scope	Pass Criteria
API Testing	Postman / browser requests	Auth, products, orders, admin endpoints	Correct 2xx/4xx responses and JSON output
UI Testing	Manual browser testing	Navigation, forms, cart, checkout, admin pages	No broken pages or incorrect flows
Authentication Testing	Manual login/register tests	JWT token, protected routes, admin-only access	Users cannot access admin actions without admin role

Responsiveness	Chrome DevTools device toolbar	Mobile and desktop widths	No major layout breaks
Deployment Testing	Render live URL testing	Frontend/backend connection and MongoDB persistence	Products and API data load correctly from the Render deployment
Database Testing	MongoDB Atlas / Compass and API tests	Users, products, and orders collections	Data is saved, updated, retrieved, and deleted correctly through the API

9.6 Deployment & DevOps

- Version control is managed using Git and GitHub.
- The project was deployed using Render so the website can be accessed online from one hosted service for the live demo.
- For Render deployment, server.js uses process.env.PORT and express.static to serve the frontend folder from the Express application.
- Environment-sensitive values such as JWT secrets and the MongoDB connection string (MONGO_URI) should be moved to environment variables for production-level security.
- Final demo should confirm that product loading, login, cart, checkout, admin functions, and MongoDB data persistence work from the hosted URL.

10. Submission Checklist

- ☒ System architecture diagram completed
- ☒ Database ERD finalized
- ☒ All API endpoints documented
- ☒ Frontend pages implemented and responsive
- ☒ Backend API tested and returning correct responses
- ☒ Authentication and authorization working correctly
- ☒ Frontend and backend integrated
- ☒ Database seeded with sample data
- ☒ Testing plan prepared
- ☒ Application deployed on Render
- ☒ Environment variables reviewed
- ☒ Code pushed to GitHub
- ☒ Final project report completed and formatted
- ☒ Presentation slides prepared
- ☒ Live demo rehearsed by all team members